Assignment 1, Digital Signal Processing:

Fourier Transform

University of Glasgow, School of Engineering

Yinan LU * 2455551L

Q1. Record the voice of one member of your group as uncompressed WAV. Say one complete sentence. It can be any language. Make sure that you record at least at 44kHz or at a higher sampling rate and that the audio is not clipping. The full audio spectrum up to 20 kHz needs to be available. Low quality MP3 downloads from websites are not allowed. Reports based on recordings at sampling rates below 44kHz or downloads will not be marked.

I recorded my sound in m4a with my smartphone and converted it online. (https://www.aconvert.com/cn/audio/). I set the sample rate to 44.1kHz. The original wav file is in the zip file.

Q2. Load the audio samples into python and plot the audio signal in the time domain (linear axis) and in the frequency domain (logarithmic axis for both frequency and amplitude) with proper axis labels (time, frequency, amplitude, dB).

By using matplotlib.pyplot, we can create pictures with labels and data in vector from. I divided the audio into left and right channels, analyzed their time domain and frequency domain, and drew them with pyplot. Finally, merge the two graphs into one figure.

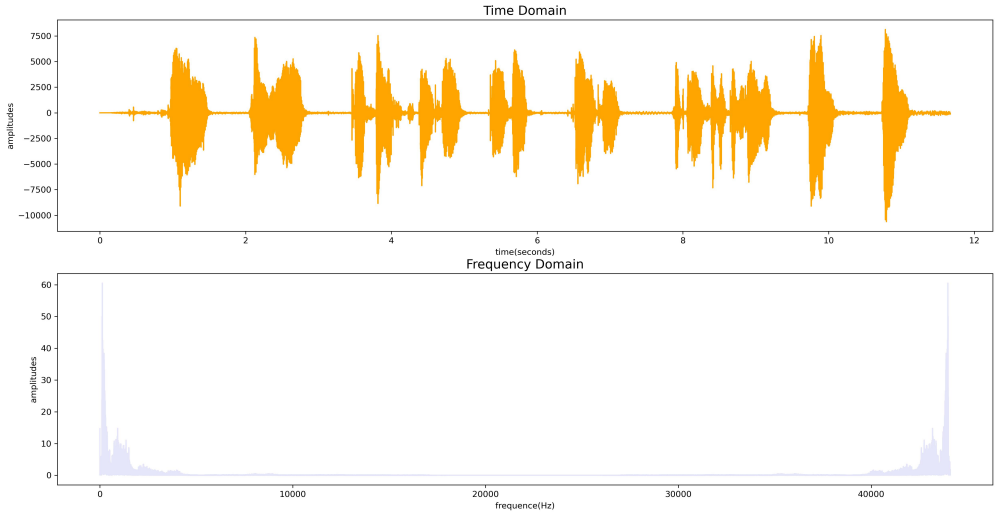


Figure 2.1 left audio signal

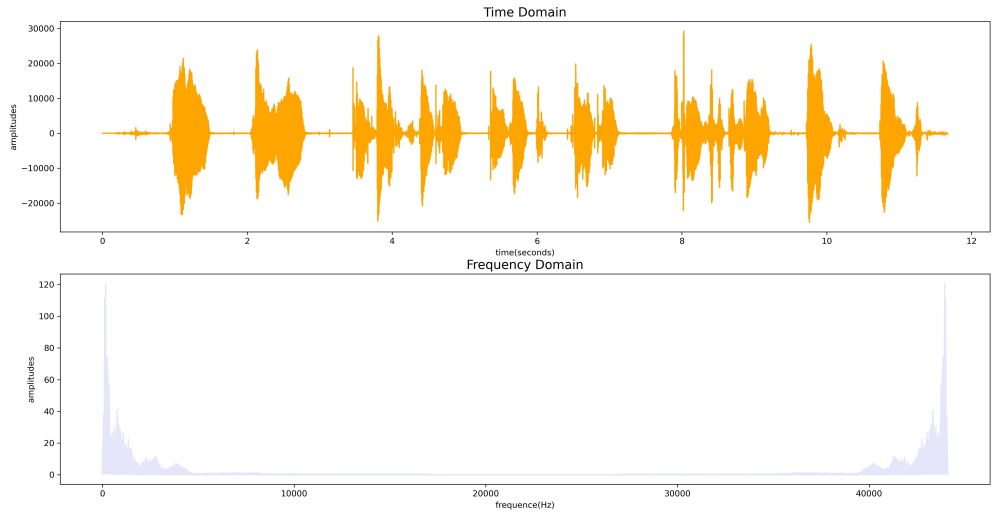


Figure 2.2 right audio signal

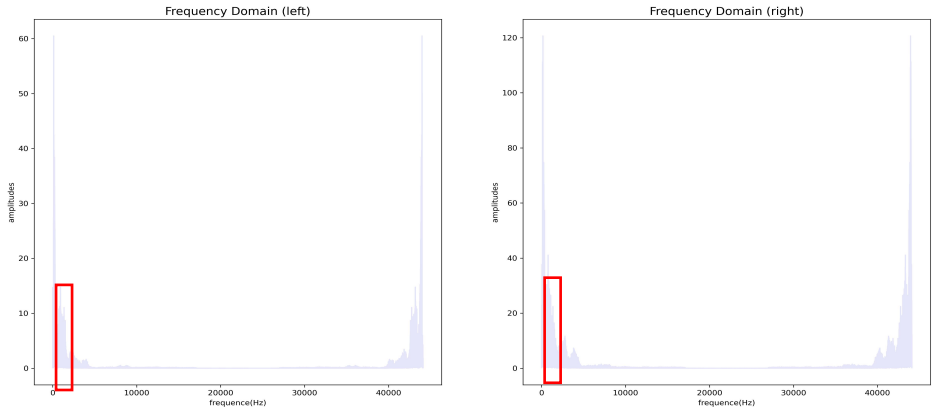
Those two functions are designed to get time domain and frequency domain. Using wave.open(), we can get the number of channel, sample width, frame rate and frame numbers. Then we can put data into arrays and get time domain and frequency domain by fast fourier transform. Finally, plot the figures.

```
def get_wavedata(wavfile):

    wf = wave.open(wavfile, "rb")       # open wav
    params = wf.getparams()         # get parameters
    nchannels, sampwidth, framerate, nframes = params[:4]
```

```
    str_data = wf.readframes(nframes)     # read whole frame data to str_data
    wf.close()   # close wave

    # transform wave data to array
    wave_data = np.frombuffer(str_data, dtype=np.short)
    wave_data.shape = -1, 2
    wave_data = wave_data.T
    return wave_data

def get_timedomain(wavfile):

    wave_data = get_wavedata(wavfile)   # Get processed wave data
    nchannels, sampwidth, framerate, nframes=get_wave_params(wavfile)

    # Construct abscissa
    time_length = np.arange(0, nframes) * (1.0 / framerate)
    return time_length, wave_data[0], wave_data[1]

def get_freqdomain(wavfile):

    waveData = get_wavedata(wavfile)   # get wave data
    nchannels, sampwidth, framerate, nframes = get_wave_params(wavfile)

    # FFT,  get amplitude
    fft_y1 = np.fft.fft(waveData[0]) / nframes   # left vocal channel
    fft_y2 = np.fft.fft(waveData[1]) / nframes   # right vocal channel

    # calculate frequency domain x
    freqs = np.linspace(0, framerate, nframes)

    return freqs, np.abs(fft_y1), np.abs(fft_y2)
```

Q3. Explain which peaks in the spectrum correspond to the fundamentals of the vowels spoken. Explain which frequency range mainly contains the consonants.

The sentence of WAV file is “Hello, how are you. I am fine. Thank you. And you? I am OK. Hope you have a good day. Bye,Bye.” So in this sentence, we can see from the below Figure that the frequencies is concentrated in 0~5000Hz and 38000~44000Hz. The fundamental frequency of human voice is between 50-500 and the fundamental frequency of vowels is 200-600 Hz. There are three vowels, which are o[əu], u[ju:], i[ai].



Q4. Improve the quality of your voice by increasing the amplitudes of the base of your voice and the region around 6 10 kHz by using the Fast Fourier Transform: transform the time domain signal into the frequency domain, then manipulate the spectrum and then transform it back to the time domain. Make sure that the audio is not clipping or distorted. Save the resulting time domain signal as a WAV (16 bit) and include it in your submission.

We can enhance the signal between 6-10 Hz to improve the quality of the audio. First, I convert the time domain signal to the frequency domain, then enhance the signal between 6-10 Hz, and then convert it back to the time domain. As the shown in the below figure, the signal between 6-10 Hz has been enhanced. As a result, the wav file which was processed is louder than the original one.

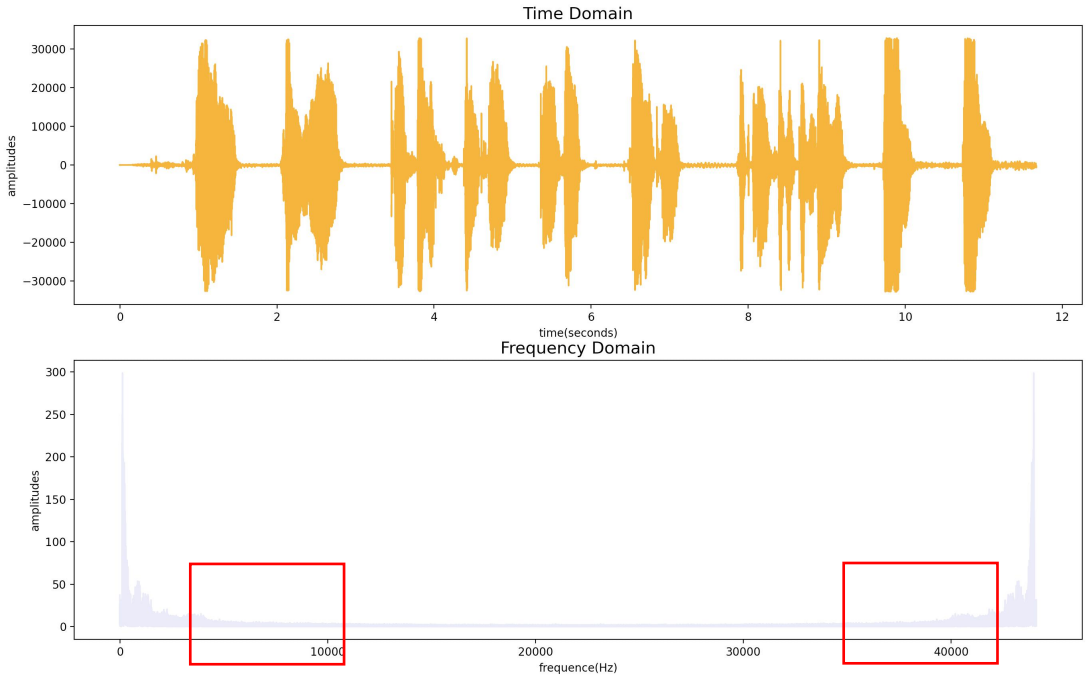


Figure 4.1 after improving(left channel)

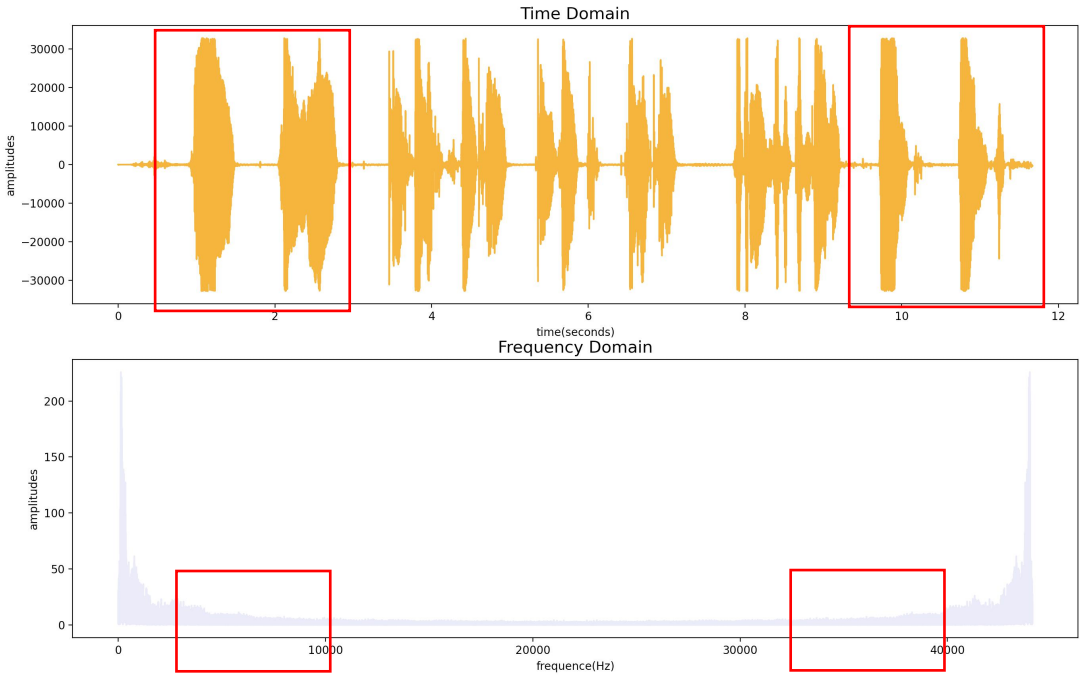


Figure 4.2 after improving (right channel)

```
# enhance tone amplitude
def amplitudes_increase(freq_data, real_value, imag_value):

    freq_data = freq_data.tolist()
    freq_data_real = [x.real for x in freq_data]
```

```
    freq_data_imag = [x.imag for x in freq_data]

    # enhance amplitude
    freq_data_real = [x*real_value for x in freq_data_real]
    freq_data_imag = [x*imag_value for x in freq_data_imag]
    freq_data_enhance = np.array([complex(x, freq_data_imag[step]) for step, x in enumerate(freq_data_real)])

    return freq_data_enhance

# 6k-10k area, filtering
def region_enhance(freq_data, framerate, nframes, enhance_region=[6000,10000]):

    half_framerate = math.ceil(framerate/2)
    symmetry_enhance_region = [(half_framerate-x)*2+x for x in enhance_region]
    enhance_region = [int(x/framerate * nframes) for x in enhance_region]
    symmetry_enhance_region = [int(x/framerate * nframes) for x in symmetry_enhance_region]

    for i in range(enhance_region[0], enhance_region[1]):
        freq_data[i] = complex(0, 0)
    for i in range(symmetry_enhance_region[1], symmetry_enhance_region[0]):
        freq_data[i] = complex(0, 0)

    return freq_data

def enhance_freq(waveData, framerate, nframes, real_value, imag_value):

    freq_data = np.fft.fft(waveData)/nframes
    # enhance amplitude
    freq_data_enhance = amplitudes_increase(freq_data, real_value, imag_value)
    # 6k-10k area, filter to remove noise
    freq_data_enhance = region_enhance(freq_data_enhance, framerate, nframes)
    # Inverse Fourier transform
    time_data = np.fft.ifft(freq_data_enhance*nframes).real.astype(np.short)

    return time_data
```

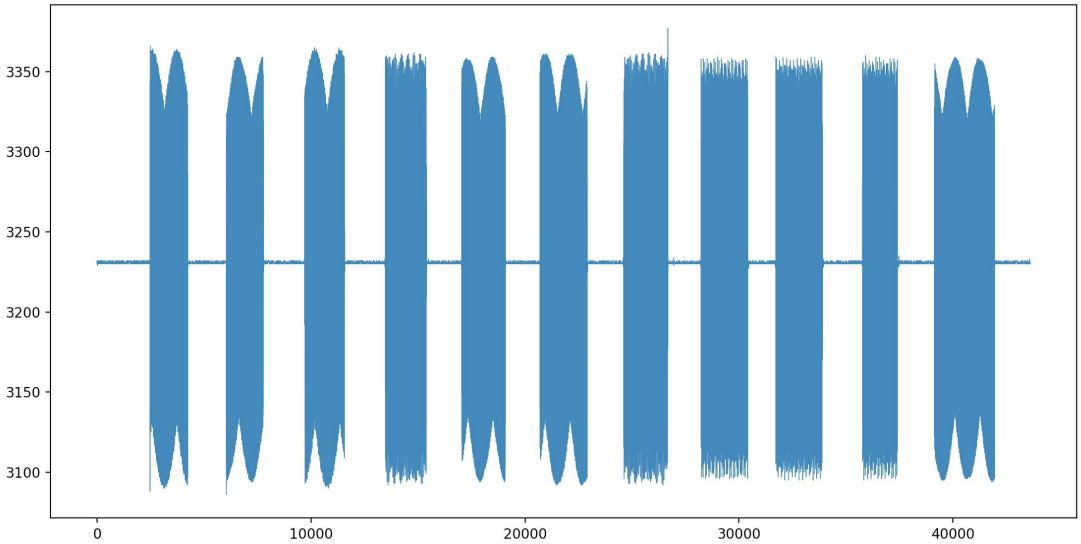
5. We have dialled numbers on a touch tone telephone and uploaded the files to moodle. There are different files depending on your matric number and class. Make sure you download the right one. Your task is to use the Fourier Transform to determine which numbers have been dialled. The Python program should be written in a way that it can detect all numbers automatically and prints them on the

screen. For that purpose write

(a) a python function which detects one digit from a chunk of the recording and then

(b) write the main program as a loop which detects the key presses and then feeds them into the python function in 5a).

My matric number is 2455551L. I had a look at the time domain of “msc_matric_1.dat” file. But I find some strange data. These data have 11 groups and they are so similar that I couldn't separate it even though I have tried many methods. I know we need to analyze it with DTMF, but I failed. So I don’t get telephone number finally. Sorry :(



Frequencies	1209 Hz	1336 Hz	1477 Hz
697 Hz	1	2	3
770 Hz	4	5	6
852 Hz	7	8	9
941 Hz	*	0	#